

Deep Learning

Transformer network

Jan Platoš, Radek Svoboda

May 5, 2026

Department of Computer Science

Faculty of Electrical Engineering and Computer Science

VŠB - Technical University of Ostrava

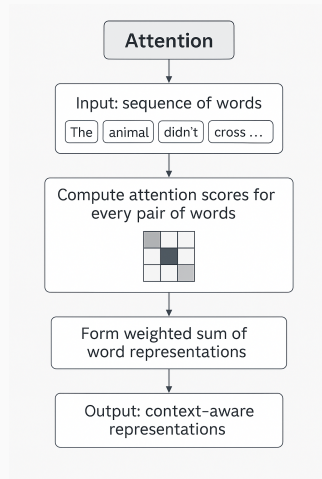
Transformer network

Transformer network

- Recurrent neural network works in synchronized serial way.
- This prevent efficient computation in parallel.
- The attention-mechanism looks at an input sequence and decides at each step which other parts of the sequence are important.
- The attention mechanism helps the decoder to focus on the important part of the input sequence.

Transformer network - Attention- What it is?

- Allows the model to **focus on relevant parts** of the input when processing each word.
- Inspired by how humans read – we don't process all words equally.
- For each word:
 - Computes how much attention it should pay to every other word in the sentence.
 - Creates a **weighted sum** of all word representations.
- Output: a **context-aware representation** of each word in the sentence.



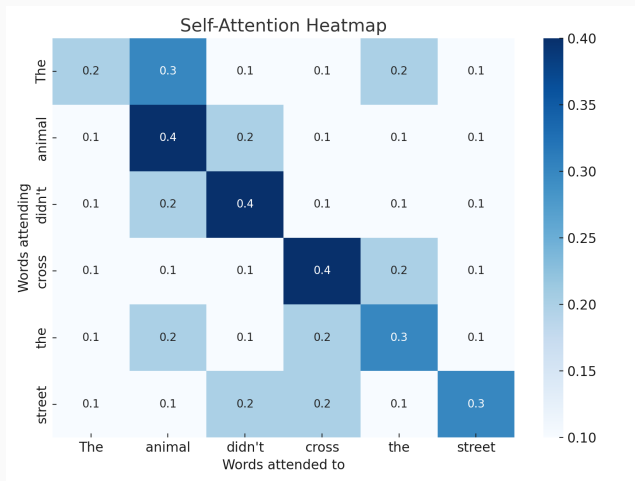
Transformer network - Query, Key, and Value

- Each input word is projected into three vectors:
 - **Query (Q)** – What we are looking for.
 - **Key (K)** – What each word offers.
 - **Value (V)** – The content/information of each word.
- For each word, compute attention weights:

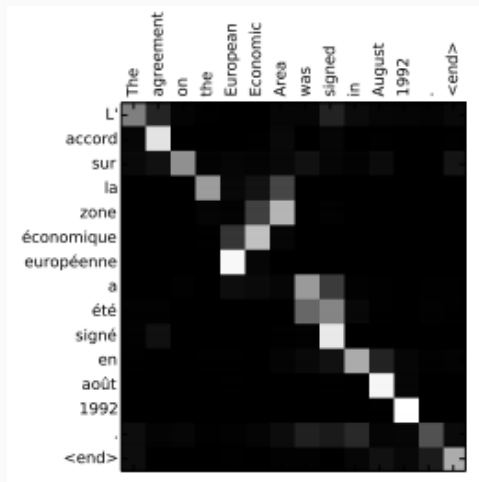
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- The result is a new representation that captures the context of the word.

Transformer network - Attention



Transformer network - Attention



- The attention allows you to look at the totality of a sentence to make connections between any particular word and its relevant context.
- For each input that the LSTM (Encoder) reads, the attention-mechanism takes into account several other inputs at the same time and decides which ones are important by attributing different weights to those inputs.
- The Decoder will then take as input the encoded sentence and the weights provided by the attention-mechanism.

Transformer network - Attention

- Transformer is a novel architecture that utilizes Attention to process Sequence2Sequence tasks.
- It consists of an Encoder and Decoder, but they are not based on recurrent connections.
- Encoder and Decoder are composed of modules that can be stacked on top of each other multiple times ($N \times$).

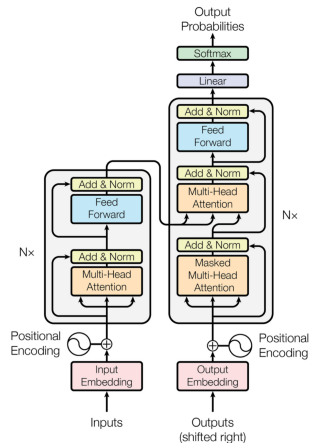


Figure 1: The Transformer - model architecture.

Transformer network - Positional encoding- What is?

- Transformers do not have recurrence or convolution.
- Therefore, they have no built-in sense of word order.
- Positional encoding injects information about the position of each token in the sequence.
- These encodings are added to the input embeddings.
- Uses sinusoidal functions to ensure unique and continuous position representation.

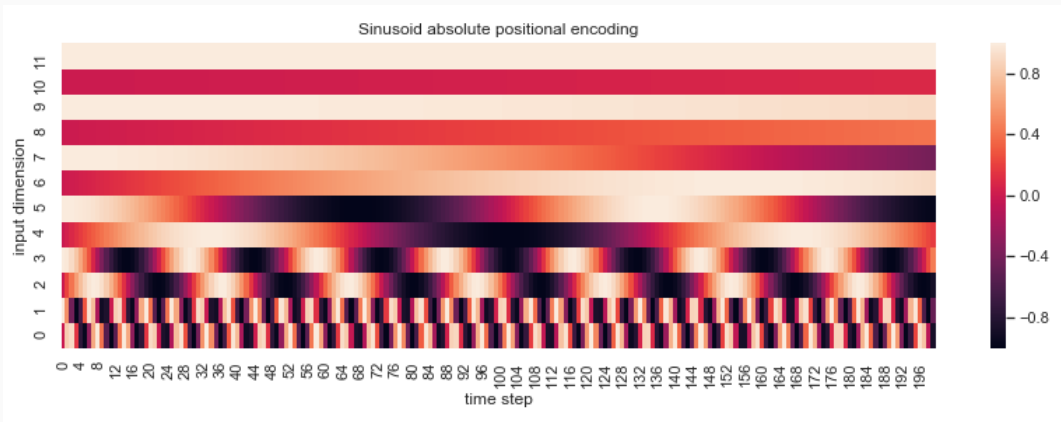
Transformer network - Positional encoding- Formula

$$\text{PE}_{(\text{pos}, 2i)} = \sin \left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}} \right)$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos \left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}} \right)$$

- pos is the position in the sequence (e.g., 0, 1, 2, ...).
- i is the dimension index.
- d_{model} is the total dimension of the embedding vector.

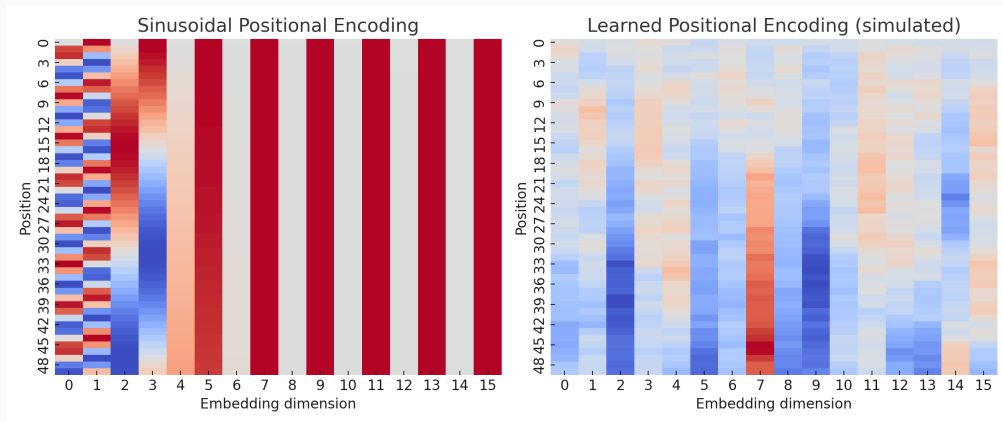
Transformer network - Positional encoding- Formula



Transformer network - Positional encoding- Notes

- **Sinusoidal functions** are used to provide:
 - Unique yet smoothly varying encoding for each position.
 - Ability to **interpolate** or generalize to longer sequences.
 - Simple mathematical patterns that can capture relative distances.
- **Alternative: learned positional encodings**
 - Positional vectors are trainable.
 - More flexible, but limited to sequence lengths seen during training.
- **Without PE:**
 - Model loses word order completely (e.g., “dog bites man” = “man bites dog”).

Transformer network - Positional encoding- Comparison



Transformer network - Multi-head Attention- What is?

- Instead of computing attention once, we do it in parallel multiple times (heads).
- Each head learns different projections of Q, K, V with its own parameters:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

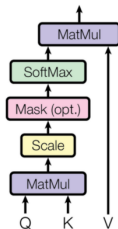
- All heads are concatenated and projected again:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

- This allows the model to jointly attend to information from different representation subspaces.

Transformer network - Multi-head Attention

Scaled Dot-Product Attention



Multi-Head Attention

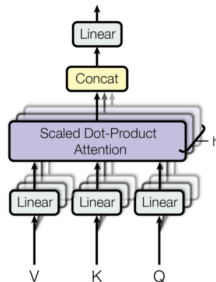


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Transformer network - Multi-head Attention

- Multi-head attention = multiple attention mechanisms in parallel.
- Each head uses its own learned projection matrices:

$$QW_i^Q, \quad KW_i^K, \quad VW_i^V$$

- Different heads can focus on different types of relationships (e.g. syntax, semantics).
- The outputs of all heads are concatenated and linearly projected:

$$\text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

- Encoder and decoder use multi-head attention differently:
 - Encoder: attends to all positions in the input.
 - Decoder: attends to previous positions (masked).
 - Encoder-decoder: decoder attends to encoder output.

Training a Transformer (Seq2Seq model):

- Uses **teacher forcing** – the true previous output is fed into the decoder.
- The target sequence is **right-shifted** by one position.
- A **<sos>** token is added at the beginning, and **<eos>** at the end.
- A **causal mask** ensures decoder only sees previous positions.
- Loss is computed between decoder output and original target sequence.

- The target sequence is **right-shifted** to prevent the model from copying input.
- A **<sos>** token is added at the beginning, and **<eos>** at the end.
- A **causal mask** ensures that each decoder step sees only previous tokens.
- The loss is computed against the original (unshifted) target sequence.

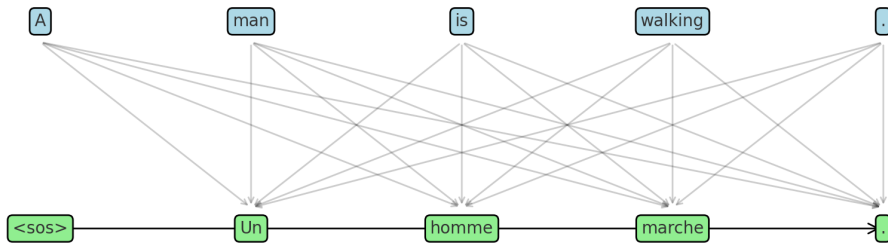
Inference – autoregressive decoding:

1. Input the full encoder sequence.
2. Start decoder with `<sos>` token.
3. Predict the first output token.
4. Append it to the decoder input and repeat.
5. Stop when the model predicts `<eos>`.

At each step, the decoder sees its own past outputs and the encoder context.

Transformer network - Inference

Transformer Inference (Autoregressive Decoding)



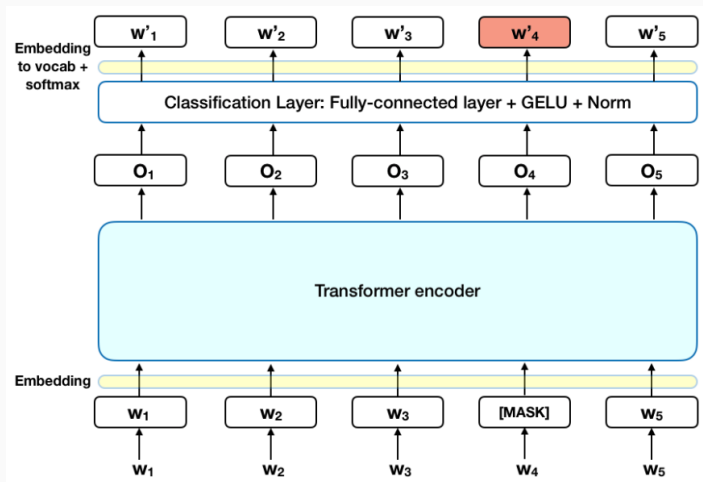
Transformer network - BERT

- BERT - Bidirectional Encoder Representations from Transformers.
- Uses a Transformer-based network with pre-trained deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers.
- BERT makes use of Transformer, an attention mechanism that learns contextual relations between words (or sub-words) in a text.
- Transformer includes two separate mechanisms — an encoder that reads the text input and a decoder that produces a prediction for the task. Since BERT's goal is to generate a language model, only the encoder mechanism is necessary.

Transformer network - BERT

- As opposed to directional models, which read the text input sequentially (left-to-right or right-to-left), the Transformer encoder reads the entire sequence of words at once.
- Therefore it is considered bidirectional, though it would be more accurate to say that it's non-directional. This characteristic allows the model to learn the context of a word based on all of its surroundings (left and right of the word).
- The input is a sequence of tokens, which are first embedded into vectors and then processed in the neural network.
- The output is a sequence of vectors of size H , in which each vector corresponds to an input token with the same index.
- When training language models, there is a challenge of defining a prediction goal.

Transformer network - BERT



Transformer network - BERT - Masked LM

- Before feeding word sequences into BERT, 15% of the words in each sequence are replaced with a [MASK] token.
- The model then attempts to predict the original value of the masked words, based on the context provided by the other, non-masked, words in the sequence.
- In technical terms, the prediction of the output words requires:
 1. Adding a classification layer on top of the encoder output.
 2. Multiplying the output vectors by the embedding matrix, transforming them into the vocabulary dimension.
 3. Calculating the probability of each word in the vocabulary with softmax.
- The BERT loss function takes into consideration only the prediction of the masked values and ignores the prediction of the non-masked words.

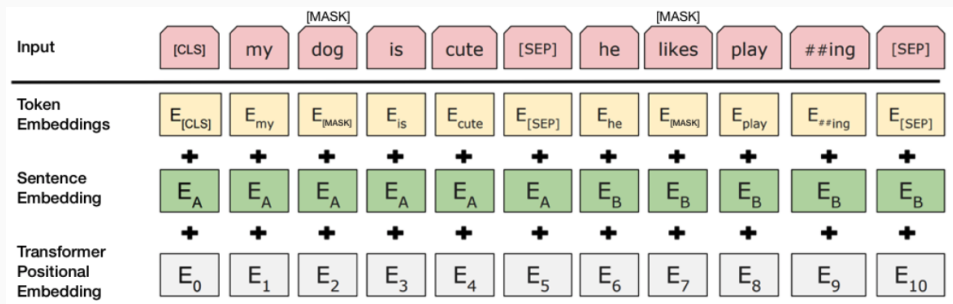
Transformer network - BERT - Next Sentence Prediction (NSP)

- In the BERT training process, the model receives pairs of sentences as input and learns to predict if the second sentence in the pair is the subsequent sentence in the original document.
- During training, 50% of the inputs are a pair in which the second sentence is the subsequent sentence in the original document, while in the other 50% a random sentence from the corpus is chosen as the second sentence.
- The assumption is that the random sentence will be disconnected from the first sentence.

Transformer network - BERT - Next Sentence Prediction (NSP)

- To help the model distinguish between the two sentences in training, the input is processed in the following way before entering the model:
 1. A [CLS] token is inserted at the beginning of the first sentence and a [SEP] token is inserted at the end of each sentence.
 2. A sentence embedding indicating Sentence A or Sentence B is added to each token. Sentence embeddings are similar in concept to token embeddings with a vocabulary of 2.
 3. A positional embedding is added to each token to indicate its position in the sequence. The concept and implementation of positional embedding are presented in the Transformer paper.

Transformer network - BERT - Next Sentence Prediction (NSP)



Transformer network - BERT - Next Sentence Prediction (NSP)

- To predict if the second sentence is indeed connected to the first, the following steps are performed:
 1. The entire input sequence goes through the Transformer model.
 2. The output of the [CLS] token is transformed into a 2×1 shaped vector, using a simple classification layer (learned matrices of weights and biases).
 3. Calculating the probability of IsNextSequence with softmax.
- When training the BERT model, Masked LM and Next Sentence Prediction are trained together, with the goal of minimizing the combined loss function of the two strategies.

Transformer network - BERT - How to use BERT (Fine-tuning)

- BERT can be used for a wide variety of language tasks, while only adding a small layer to the core model:
 1. Classification tasks such as sentiment analysis are done similarly to Next Sentence classification, by adding a classification layer on top of the Transformer output for the [CLS] token.
 2. In Question Answering tasks (e.g. SQuAD v1.1), the software receives a question regarding a text sequence and is required to mark the answer in the sequence. Using BERT, a Q&A model can be trained by learning two extra vectors that mark the beginning and the end of the answer.
 3. In Named Entity Recognition (NER), the software receives a text sequence and is required to mark the various types of entities (Person, Organization, Date, etc) that appear in the text. Using BERT, a NER model can be trained by feeding the output vector of each token into a classification layer that predicts the NER label.

1. What is a Transformer? by Maxime,
[https://medium.com/inside-machine-learning/
what-is-a-transformer-d07dd1fbec04](https://medium.com/inside-machine-learning/what-is-a-transformer-d07dd1fbec04)
2. A Beginner's Guide to Attention Mechanisms and Memory Networks [https:
//wiki.pathmind.com/attention-mechanism-memory-network](https://wiki.pathmind.com/attention-mechanism-memory-network)
3. What is Teacher Forcing for Recurrent Neural Networks? by Jason Brownlee
[https://machinelearningmastery.com/
teacher-forcing-for-recurrent-neural-networks/](https://machinelearningmastery.com/teacher-forcing-for-recurrent-neural-networks/)
4. Positional Encoding: Everything You Need to Know by Darjan Salaj
[https://www.inovex.de/de/blog/
positional-encoding-everything-you-need-to-know](https://www.inovex.de/de/blog/positional-encoding-everything-you-need-to-know)

5. Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning. "Effective approaches to attention-based neural machine translation." arXiv preprint arXiv:1508.04025 (2015).
6. Devlin, Jacob, et al. "Bert: Pre-training of deep bidirectional transformers for language understanding." arXiv preprint arXiv:1810.04805 (2018).
7. BERT Explained: State of the art language model for NLP by Rani Horev
<https://towardsdatascience.com/bert-explained-state-of-the-art-language-model-for-nlp-f8b21a>
8. Understanding BERT Transformer: Attention isn't all you need by Damien Sileo <https://medium.com/synapse-dev/understanding-bert-transformer-attention-isnt-all-you-need-58>

Questions?